

Simulación de la Ecuación del Calor 3D en Estado Estacionario con Paralelización OpenMP

Gabriel Coto Fernández (C4E540)
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
gabriel.cotofernandez@ucr.ac.cr

Jhon Campos Montenegro (C4D674)
Escuela de Ciencias de la Computación e Informática
Universidad de Costa Rica
San José, Costa Rica
jhon.campos@ucr.ac.cr

Abstract—Se presenta una implementación de la ecuación del calor 3D en estado estacionario usando diferencias finitas con estencil de 6 vecinos sobre una malla $N \times N \times N$, en versiones secuencial y paralela con OpenMP en C++. Los experimentos de escalamiento fuerte y débil se ejecutaron en el clúster Kabré del CENAT con mallas de $N=100$, 200 y 300, y hasta 16 hilos. Para $N=100$, la paralelización con `collapse(2)` y `schedule(static)` alcanza un speedup de $4.04 \times$ con 8 hilos (eficiencia 50 %). Para mallas grandes ($N \geq 200$), el cuello de botella se desplaza al ancho de banda de memoria DRAM: el speedup se estanca cerca de $1.8\text{--}2.1 \times$ independientemente del número de hilos, lo que evidencia saturación del bus de memoria en el nodo de cómputo. El escalamiento débil muestra eficiencia del 97.5 % con 2 hilos, pero colapsa al 56 % con 4 hilos debido a accesos inter-socket (NUMA).

Index Terms—OpenMP, ecuación del calor, diferencias finitas, HPC, Kabré, escalamiento, paralelismo de datos

I. INTRODUCCIÓN

La ecuación de Laplace y la ecuación de Poisson son las ecuaciones diferenciales parciales (EDPs) de segundo orden que modelan fenómenos físicos de equilibrio: distribución de temperatura, potencial eléctrico y presión hidrostática, entre otros. En estado estacionario, la distribución de temperatura $u(x, y, z)$ en un dominio cúbico satisface la ecuación de Poisson:

$$u_{xx} + u_{yy} + u_{zz} = f(x, y, z) \quad (1)$$

Para el caso homogéneo ($f = 0$), la solución se obtiene mediante el método de diferencias finitas con el estencil de 7 puntos (6 vecinos en 3D):

$$w_{i,j,k} = \frac{w_{i\pm 1,j,k} + w_{i,j\pm 1,k} + w_{i,j,k\pm 1}}{6} \quad (2)$$

Este tipo de cálculo iterativo se presta naturalmente a la paralelización, pues cada punto interior puede actualizarse de forma independiente dado que los valores de entrada son fijos durante cada paso.

El objetivo de este trabajo es comparar el rendimiento de la versión secuencial contra versiones paralelas con distintas cantidades de hilos OpenMP, y cuantificar la ganancia obtenida a través de métricas estándar de cómputo de alto rendimiento (HPC).

II. DISEÑO E IMPLEMENTACIÓN

A. Estructura modular del código

El proyecto se organizó en los siguientes módulos:

- **grid.h / grid.cpp**: clase Grid que encapsula la malla 3D en un `std::vector<double>` contiguo (layout row-major). Provee los métodos `get/set` inlineados y la función `swap_data` de complejidad $O(1)$.
- **solver.h / solver.cpp**: versión secuencial del estencil y del ciclo de simulación con doble buffering.
- **Paralelo**: `parallel.h` y `parallel.cpp` contienen la versión con directivas OpenMP.
- **output.h / output.cpp**: exportación del estado final al formato VTK para visualización en ParaView.
- **Temporizador**: `timer.h` define `now()` sobre `omp_get_wtime()`.
- **main.cpp**: orquestación, medición y reporte de tiempos.

B. Representación de la malla

La malla se representa como un vector plano de tamaño N^3 , donde el índice lineal de (i, j, k) es:

$$\text{idx}(i, j, k) = i \cdot N^2 + j \cdot N + k \quad (3)$$

Esta elección favorece la localidad de caché frente a la alternativa de punteros múltiples (`double***`), que introduciría niveles adicionales de indirección.

C. Condiciones de frontera e inicialización

Las condiciones de frontera son:

- Cara posterior ($j = N - 1$): $T = 0.0^\circ\text{C}$
- Las demás cinco caras: $T = 100.0^\circ\text{C}$

Los puntos interiores se inicializan con el promedio de las temperaturas de frontera:

$$T_{\text{init}} = \frac{5 \times 100.0 + 1 \times 0.0}{6} \approx 83.33^\circ\text{C} \quad (4)$$

D. Doble buffering

Se emplean dos mallas (`old` y `new`) que alternan roles en cada iteración mediante intercambio de punteros. Esto evita copias $O(N^3)$ entre pasos; en cambio, el intercambio es $O(1)$. Al finalizar, si el número de pasos es impar, los buffers internos se intercambian para garantizar que el resultado siempre quede en `old_grid`.

E. Paralelización con OpenMP

La región crítica de cómputo es el triple bucle del estencil. La directiva utilizada es:

```
#pragma omp parallel for collapse(2) schedule(
static)
for (int i = 1; i < N - 1; i++) {
  for (int j = 1; j < N - 1; j++) {
    for (int k = 1; k < N - 1; k++) {
      double sum =
        old_grid.get(i+1,j,k) + old_grid.get(i
        -1,j,k) +
        old_grid.get(i,j+1,k) + old_grid.get(i
        ,j-1,k) +
        old_grid.get(i,j,k+1) + old_grid.get(i
        ,j,k-1);
      new_grid.set(i, j, k, sum / 6.0);
    }
  }
}
```

Las decisiones de diseño son:

- **collapse(2)**: fusiona los bucles de i y j en un espacio de iteración de tamaño $(N - 2)^2$, dando mayor granularidad para balancear la carga entre hilos.
- **schedule(static)**: apropiado porque el trabajo por iteración es uniforme (siempre 6 lecturas y 1 escritura).
- **Ausencia de condiciones de carrera**: cada hilo escribe en una celda única de `new_grid`, mientras lee de `old_grid` que permanece constante durante la iteración.

F. Salida para ParaView

La función `write_vtk` genera un archivo en formato *VTK Legacy ASCII*, con tipo de datos `STRUCTURED_POINTS`. Este formato es importado directamente por ParaView sin necesidad de plugins adicionales.

III. METODOLOGÍA DE PRUEBAS

A. Entorno de ejecución: clúster Kabré

Todas las mediciones de rendimiento se realizaron en el clúster Kabré del CENAT (Centro Nacional de Alta Tecnología) de Costa Rica. Kabré es un sistema HPC con nodos de cómputo equipados con procesadores Intel Xeon de múltiples núcleos, interconectados mediante InfiniBand. Para garantizar reproducibilidad, todas las corridas se ejecutaron en el mismo tipo de nodo, sin otras cargas de usuario activas, usando el sistema de colas SLURM.

El código fue compilado con:

```
g++ -O2 -fopenmp -o heat main.cpp grid.cpp
...
```

B. Justificación de los casos de prueba

Los parámetros del experimento se escogieron para capturar tres fenómenos distintos:

1) Selección de N (tamaño de malla):

- $N = 100$: tamaño base definido en el enunciado (10^6 celdas). Sirve como punto de referencia y para medir si el overhead de OpenMP supera el trabajo útil en problemas pequeños.
- $N = 200$: requerido por la rúbrica para evaluar throughput (8×10^6 celdas). Comienza a saturar la caché L3 de los nodos, haciendo la paralelización más beneficiosa.
- $N = 300$: escenario de competición (2.7×10^7 celdas). Produce presión de memoria significativa y pone de manifiesto limitaciones de ancho de banda.

2) *Selección de hilos*: Se probaron 1, 2, 4, 8 y 16 hilos. La elección sigue una progresión en potencias de 2, estándar en estudios HPC porque facilita el cálculo del speedup ideal y revela quiebres de eficiencia asociados a la topología NUMA y el tamaño de los sockets.

3) *Selección de NUM_STEPS*: Se fijó en 1000 iteraciones para todas las corridas de escalamiento, siguiendo la especificación del enunciado. Esto asegura que la simulación llega suficientemente cerca del estado estacionario y que el tiempo de cómputo es representativo (domina sobre el tiempo de inicialización y E/S).

C. Métricas

1) Speedup (S):

$$S(p) = \frac{T_{\text{seq}}}{T_{\text{par}}(p)} \quad (5)$$

donde p es la cantidad de hilos.

2) Eficiencia (E):

$$E(p) = \frac{S(p)}{p} \times 100 \% \quad (6)$$

3) *Escalamiento fuerte (Ley de Amdahl)*: Se mantiene el problema fijo (N y `NUM_STEPS` constantes) y se varía p . El speedup máximo teórico está limitado por la fracción secuencial α del programa:

$$S_{\text{max}} = \frac{1}{\alpha + \frac{1-\alpha}{p}} \quad (7)$$

En nuestra implementación, la fracción secuencial está compuesta por la inicialización de fronteras, el swap de punteros y la escritura VTK.

4) *Escalamiento débil (Ley de Gustafson)*: Se mantiene la carga por hilo constante: si p hilos procesan una malla de tamaño N_0^3 , entonces 1 hilo procesaría N_0^3/p puntos. La métrica de escalamiento débil es:

$$\text{Eficiencia débil}(p) = \frac{T_1(N_0/\sqrt[3]{p})}{T_p(N_0)} \times 100 \% \quad (8)$$

IV. RESULTADOS

A. Escalamiento fuerte

Las tablas I–III presentan los tiempos promedio de tres corridas independientes en Kabré, junto con el speedup $S(p) = T_{\text{seq}}/T_{\text{par}}(p)$ y la eficiencia $E(p) = S(p)/p \times 100 \%$ correctamente calculados.

TABLE I
ESCALAMIENTO FUERTE – N=100, 1000 PASOS

Hilos	T_{seq} (s)	T_{par} (s)	$S(p)$	$E(p)$ (%)
1	1.350	1.387	0.973	97.33
2	1.353	0.692	1.955	97.77
4	1.334	0.403	3.308	82.70
8	1.335	0.331	4.035	50.44
16	1.346	0.492	2.737	17.11

TABLE II
ESCALAMIENTO FUERTE – N=200, 1000 PASOS

Hilos	T_{seq} (s)	T_{par} (s)	$S(p)$	$E(p)$ (%)
1	14.369	14.432	0.996	99.56
2	14.398	12.094	1.190	59.52
4	14.383	8.241	1.745	43.63
8	14.329	7.909	1.812	22.65
16	14.299	7.899	1.810	11.31

B. Escalamiento débil

Se mantuvo una carga de $\approx N_0^3 = 10^6$ puntos interiores por hilo, escalando N según $N = \lfloor 100 \cdot p^{1/3} \rfloor$. La eficiencia débil se define como $E_{\text{débil}}(p) = T_{\text{par}}(1, N_0) / T_{\text{par}}(p, N_0 \cdot p^{1/3}) \times 100\%$, es decir, el tiempo base con 1 hilo dividido entre el tiempo paralelo con p hilos procesando la carga proporcionalmente mayor.

C. Análisis de resultados

Los resultados revelan dos regímenes de comportamiento claramente distintos según el tamaño de la malla.

Malla pequeña (N=100). La paralelización funciona bien hasta 8 hilos, alcanzando un speedup de $4.03\times$ con eficiencia del 50%. Con 4 hilos la eficiencia es del 82.7%, cercana al ideal. El caso de 16 hilos muestra una *regresión*: el speedup baja de $4.03\times$ a $2.74\times$. Esto ocurre porque la malla posee solo $(N-2)^3 \approx 941,192$ puntos interiores; con `collapse(2)` el espacio de iteración fusionado es de $98 \times 98 = 9,604$ chunks, que distribuidos entre 16 hilos resultan en ~ 600 iteraciones por hilo, insuficientes para amortizar el overhead de gestión de hilos de OpenMP.

Mallas grandes (N=200 y N=300). El patrón es radicalmente distinto y revela un cuello de botella de *ancho de banda de memoria*. Para N=200, el tiempo paralelo se estanca en ≈ 7.9 s tanto con 8 como con 16 hilos (speedup prácticamente idéntico: 1.812 vs. 1.810). Para N=300, agregar más allá de 4 hilos es contraproducente: el speedup *decrece* de $2.14\times$ con 4 hilos a $2.03\times$ con 8 hilos y $1.67\times$ con 16.

Este comportamiento es consistente con la saturación del bus de memoria DRAM del nodo. La malla de N=200 requiere dos buffers de `double`, ocupando $200^3 \times 2 \times 8 \approx 512$ MB, muy por encima de la caché L3 típica de los nodos de Kabré. El estencil de 6 vecinos realiza solo ~ 7 operaciones flotantes por cada 6 lecturas y 1 escritura de `double` (56 bytes), lo que resulta en una intensidad aritmética de $7/56 \approx 0.125$ FLOP/byte, valor extremadamente bajo que hace al programa *memory-bound* en lugar de *compute-bound*. Cuando

TABLE III
ESCALAMIENTO FUERTE – N=300, 1000 PASOS

Hilos	T_{seq} (s)	T_{par} (s)	$S(p)$	$E(p)$ (%)
1	46.919	46.988	0.999	99.85
2	46.701	39.253	1.190	59.49
4	46.987	21.933	2.142	53.56
8	47.273	23.241	2.034	25.43
16	46.993	28.126	1.671	10.44

TABLE IV
ESCALAMIENTO DÉBIL – $\approx 10^6$ PUNTOS/HILO, 1000 PASOS

Hilos	N	T_{par} (s)	$E_{\text{débil}}$ (%)
1	100	1.357	100.00
2	125	1.392	97.53
4	158	2.407	56.38
8	200	7.998	16.97
16	251	14.621	9.28

todos los hilos saturan el bus, agregar más hilos no incrementa el throughput sino que aumenta la contención.

El escalamiento débil confirma el mismo efecto: la eficiencia cae abruptamente del 97.5% con 2 hilos al 56.4% con 4 hilos. Esto coincide con el momento en que OpenMP comienza a asignar hilos a núcleos de un segundo socket NUMA, cuyos accesos a memoria atraviesan un bus inter-socket de mayor latencia y menor ancho de banda efectivo.

V. VISUALIZACIÓN CON PARAVIEW

El programa genera un archivo `heat.vtk` en formato *Legacy VTK ASCII* con tipo `STRUCTURED_POINTS`. Para visualizarlo en ParaView:

- 1) **File** → **Open** y seleccionar `heat.vtk`.
- 2) En el panel *Properties*, hacer clic en **Apply**.
- 3) Cambiar la representación a *Surface* y la variable activa a **temperature**.
- 4) Aplicar el filtro *Slice* para cortes en los tres planos principales (*xy*, *xz*, *yz*).
- 5) Usar el mapa de color *Cool to Warm* para resaltar el gradiente de temperatura entre la cara fría (0°C) y las caras calientes (100°C).

VI. DESAFÍOS Y POSIBLES MEJORAS

A. Desafíos encontrados

- **Cuello de botella de memoria para N grande:** el hallazgo más relevante del experimento fue que, para $N \geq 200$, el programa es *memory-bound* con intensidad aritmética de apenas 0.125 FLOP/byte. Agregar hilos más allá del punto de saturación del bus DRAM no produce beneficio alguno.
- **Degradación por NUMA:** el colapso abrupto en el escalamiento débil al pasar de 2 a 4 hilos (de 97.5% a 56.4%) sugiere que OpenMP asigna los hilos adicionales a núcleos de un segundo socket, cuyos accesos cruzados de memoria introducen latencia extra.

Mitigación posible: usar `OMP_PROC_BIND=close` y `OMP_PLACES=cores` para forzar afinidad de hilos.

- **Regresión con 16 hilos en N=100:** el overhead de gestión de OpenMP supera el beneficio cuando la granularidad por hilo es demasiado baja (~ 600 iteraciones). El programa no debería usar más hilos que los que permiten mantener al menos ~ 5000 iteraciones por hilo en el espacio fusionado.
- **Error en la fórmula de eficiencia del script de medición:** el script original reportó $E = S \times 100$ en lugar de $E = (S/p) \times 100$, produciendo valores artificialmente altos. Los valores corregidos se presentan en las tablas de esta sección.
- **Gestión de doble buffering con número impar de pasos:** fue necesario rastrear en cuál buffer queda el resultado final y hacer un swap de datos $O(1)$ cuando es necesario, para garantizar una API consistente hacia el módulo de salida.

B. Posibles mejoras

- **Afinidad NUMA:** usar `OMP_PROC_BIND=close` asigna hilos a núcleos cercanos, reduce accesos inter-socket y puede recuperar eficiencia en $N=200$ con 4+ hilos.
- **Blocking/tiling por caché:** reorganizar el acceso al estencil en bloques de tamaño tal que quepan en L2/L3 reduce los fallos de caché. Para $N=300$, tiling de $32 \times 32 \times 32$ reduciría la presión sobre DRAM.
- **Convergencia:** terminar cuando $\max_{i,j,k} |w^{(n+1)} - w^{(n)}| < \epsilon$, en vez de usar un número fijo de pasos, evita iteraciones innecesarias en mallas pequeñas.
- **MPI + OpenMP híbrido:** para escalar a múltiples nodos de Kabré, combinar descomposición de dominio con MPI entre nodos y OpenMP dentro de cada nodo eliminaría la contención del bus de memoria al distribuir la malla físicamente entre nodos distintos.

VII. CONCLUSIONES

Se implementó correctamente la simulación de la ecuación del calor 3D mediante diferencias finitas iterativas en C++ con doble buffering y exportación VTK para ParaView. La paralelización con OpenMP usando `collapse(2)` y `schedule(static)` es efectiva para mallas pequeñas: con $N=100$ se obtiene un speedup de $4.03\times$ con 4-8 hilos (eficiencia 50–83 %).

El hallazgo central es que para mallas grandes ($N \geq 200$) el programa es *memory-bound*: la intensidad aritmética del estencil de 6 vecinos (≈ 0.125 FLOP/byte) satura el ancho de banda de DRAM antes de agotar la capacidad de cómputo de los núcleos. En este régimen, el speedup se estanca cerca de $1.8\text{--}2.1\times$ independientemente del número de hilos, y agregar más allá de 4 hilos produce rendimientos decrecientes o incluso regresiones. El escalamiento débil confirma la misma limitación, con eficiencia que colapsa al cruzar el límite de un socket NUMA.

Estos resultados muestran que la optimización de este tipo de simulaciones en sistemas de memoria distribuida requiere técnicas más allá de OpenMP puro: afinidad NUMA explícita, tiling de caché, y en última instancia un modelo MPI+OpenMP híbrido para distribuir la malla entre múltiples nodos.

REFERENCES

- [1] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM, 2007.
- [2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021. [Online]. Available: <https://www.openmp.org/specifications/>
- [3] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- [4] Centro Nacional de Alta Tecnología (CENAT), “Kabré: Clúster de Alto Rendimiento”, 2024. [Online]. Available: <https://kabre.cenat.ac.cr/>
- [5] A. Henderson, *The ParaView Guide: A Parallel Visualization Application*. Kitware, 2007.